Diseño Secuencial

Cristian Miguel Jimenez
Mecatrónica
ITLA
La Caleta, Santo Domingo
20240712@itla.edu.do

Resumen: En esta práctica se desarrolló un circuito secuencial utilizando distintos tipos de *flip-flops*, con el propósito de comprobar su funcionamiento e implementarlo físicamente en una FPGA mediante programación en Verilog.

Abstract: In this practice, a sequential circuit was developed using different types of flip-flops in order to verify its operation and implement it physically on an FPGA through Verilog programming.

Palabras clave: Flip-flop, circuito secuencial..

I. Introducción

Durante el siguiente documento se estará realizando un circuito secuencial haciendo uso de los distintos tipos de flip-flop y llevándolos a la practica mediante simulaciones e implementación en físico con FPGA.

II. Marco Teórico

Circuito secuencial: Un circuito secuencial es un tipo de circuito digital en el que la salida depende tanto de las entradas actuales como de los estados previos del sistema. Esto es posible gracias al uso de elementos de memoria, como los flip-flops, que permiten almacenar información y hacer que el circuito recuerde estados anteriores, posibilitando funciones como conteo, almacenamiento de datos o ejecución de secuencias.

Flip-flop: Un flip-flop es un dispositivo electrónico de memoria que puede almacenar un bit de información. Su estado de salida cambia según las señales de control que recibe, como pulsos de reloj o entradas lógicas. Es un componente esencial en los circuitos secuenciales y en sistemas digitales como contadores y registros.

Montaje experimental Para este laboratorio se estuvo utilizando:

1. Resistencia 10kΩ
2. Project board
3. LEDS
4. FPGA Tang primer 25k

También se utilizaron los siguientes softwares:

- Multisim
- Gowin

III. Diseño del circuito e implementación en físico con FF tipo D

El circuito a diseñar es la siguiente:

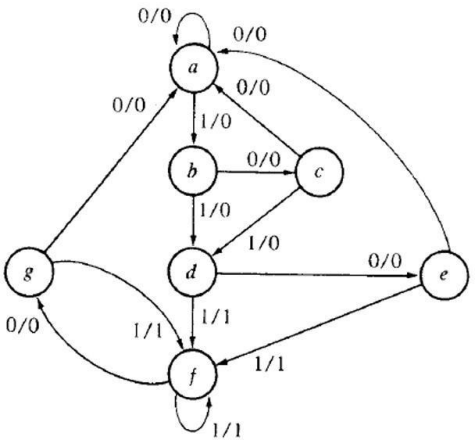


Diagrama de estados

El primer paso, es realizar la tabla de estados:

Tabla de estados							
	Actual			X	Futuro		
a	0	0	0	0	0	0	0
	0	0	0	1	0	0	1
b	0	0	1	0	0	1	0
	0	0	1	1	0	1	1
c	0	1	0	0	0	0	0
	0	1	0	1	0	1	1
d	0	1	1	0	1	0	0
	0	1	1	1	1	0	1
e	1	0	0	0	0	0	0
	1	0	0	1	1	0	1
f	1	0	1	0	1	1	0
	1	0	1	1	1	0	1
g	1	1	0	0	0	0	0
	1	1	0	1	1	0	1

Partiendo de esta tabla de estados, primero creamos la tabla de transicion con los flip-flip tipo D:

FF tipo D		
D2	D1	D0
0	0	0
0	0	1
0	1	1
0	1	0
0	1	1
0	0	1
1	0	0
1	0	1
0	0	0
1	0	1
1	1	0
1	0	1
0	0	0
1	0	1

A ahora realizamos los k-maps:

D0	00	01	11	10
00				
01			1	1
11		1		
10		1	1	1

D0 = A'BC + AC'D + AB'C

D1	00	01	11	10
00			1	1
01		1		
11				
10				1

D1= A'B'C + B'CD' + A'BC'D

D2	00	01	11	10
00		1	1	
01		1	1	
11		1		
10		1	1	

D2 = A'D + B'D + C'D

Partiendo de esas funciones, realizamos el diagrama en multisim:

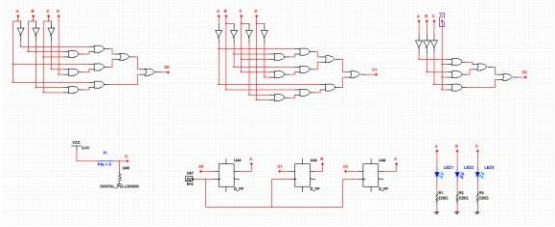


Diagrama con Flip-flop tipo D

Una vez teniendo el diagrama, podemos proceder con la escritura del codigo en verilog.

El primer paso para la escritura del codigo es escribir el modulo del Flip-flop tipo D:

```
1  module ff_D (
2     input clk,
3     input D,
4     output reg Q
5  );
6     always @(posedge clk) begin
7        Q <= D;
8     end
9  endmodule
```

Modulo de Flip-flop tipo D

Una vez con el Flip flop hecho, debemos realizar la configuracion del reloj interno del FGPA, para ello, debemos crear un divisor de reloj, y posteriormente habilitar el pin de reloj del placa.

El divisor de reloj servira para tener una frecuencia en especficico, en este caso, para la implementacion fisica, utilizaremos 1Hz.

```
module clk (
    input wire clock_in,   // Reloj de entrada
    output reg clock_out   // Reloj dividido
);

    reg [27:0] counter = 28'd0;

    // Ajusta este parámetro para obtener f_out = f_in / DIVISOR
    parameter DIVISOR = 28'd27_000_000;

    always @(posedge clock_in) begin
        // Incrementa el contador
        counter <= counter + 28'd1;

        // Cuando alcanza DIVISOR, lo reiniciamos
        if (counter >= (DIVISOR - 1))
            counter <= 28'd0;

        // Generamos un pulso de salida alto la mitad del tiempo
        clock_out <= (counter < DIVISOR/2) ? 1'b1 : 1'b0;
    end

endmodule
```

Codigo de divisor de reloj

Con esta información, se puede escribir el código utilizando lógica combinacional para complementar la conexión con los *flip-flops* y así obtener el resultado esperado.

Primero, se declaran los pines de entrada y salida; además, se utiliza la instanciación para emplear el reloj y la negación de las salidas Q, lo que facilita y hace más manejable la escritura del código.:

```
module principal (
    input wire D, clk27,
    output wire A, B, C, Y, reloj
);

//Llamamos el reloj
clk U_CLOCK(
    .clock_in(clk27),
    .clock_out(reloj)
) ;

//Negacion de Q
wire nA, nB, nC, nD ;
not U_NOTa(nA, A) ;
not U_NOTb(nB, B) ;
not U_NOTc(nC, C) ;
not U_NOTd(nD, D) ;

//Funcion de D0
wire t0, t1, t2, D0 ;
and U_AND00(t0, nA, B, C) ;
and U_AND01(t1, A, nC, D) ;
and U_AND02(t2, A, nB, C) ;
or U_OR0(D0, t0, t1, t2) ;
```

Entonces, crearemos las funciones de los Flip-flop e instancemos el flip-flop para ir implementandolo:

```
19  //Funcion de D0
20  wire t0, t1, t2, D0 ;
21  and U_AND00(t0, nA, B, C) ;
22  and U_AND01(t1, A, nC, D) ;
23  and U_AND02(t2, A, nB, C) ;
24  or U_OR0(D0, t0, t1, t2) ;
25
26  //Instanceamos FF de A y D0
27  ff_D U_FF0 (
28      .clk (reloj) ,
29      .D (D0),
30      .Q (A)
31  ) ;
32
33  //Funcion de D1
34  wire t3, t4, t5, D1 ;
35  and U_AND03(t3, nA, nB, C) ;
36  and U_AND04(t4, nB, C, nD) ;
37  and U_AND05(t5, nA, B, nC, D) ;
38  or U_OR1(D1, t3, t4, t5) ;
39
40  //Instanceamos FF de B y D1
41  ff_D U_FF1 (
42      .clk (reloj) ,
43      .D (D1),
44      .Q (B)
45  ) ;
46
47  //Funcion de D2
48  wire t6, t7, t8, D2 ;
49  and U_AND06(t6, nA, D) ;
50  and U_AND07(t7, nB, D) ;
51  and U_AND08(t8, nC, D) ;
52  or U_OR2(D2, t6, t7, t8) ;
53
54  //Instanceamos FF de B y D1
55  ff_D U_FF2 (
56      .clk (reloj) ,
57      .D (D2),
58      .Q (C)
59  ) ;
60
61  endmodule
```

IV. Desarrollo del circuito con otros Flip-flop

Primero se parte de la tabla de estados obtenida en la implementación anterior, la cual se utiliza como base para elaborar las tablas de transición correspondientes a cada tipo de flip-flop.

En primer lugar, se desarrolla el caso del *flip-flop* JK, utilizando su tabla de excitación para determinar los valores necesarios en cada transición de estado:

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$Q'(t)$

(b) JK

Tabla de exitacion el FF JK

FF Tipo JK					
J2	K2	J1	K1	J0	K0
0	X	0	X	0	X
0	X	0	X	1	X
0	X	1	X	X	0
0	X	1	X	X	1
0	X	X	0	1	X
0	X	X	1	1	X
1	X	X	1	X	1
1	X	X	1	X	0
X	1	0	X	0	X
X	0	0	X	1	X
X	0	1	X	X	1
X	0	0	X	X	0
X	1	X	1	0	X
X	0	X	1	1	X

J0	00	01	11	10
00				
01			1	1
11	x	x		
10	x	x	x	x

J0 = A'BC

K2	00	01	11	10
00	X	X	X	X
01	X	X	X	X
11	1			
10	1			

K0= C'D'

J1	00	01	11	10
00			1	1
01	X	X	X	X
11	X	X		
10				1

J1 = A'C + B'CD'

K1	00	01	11	10
00	X	X	X	X
01		1	1	1
11	1	1		
10	X	X	X	X

K1 = C'D + A'C + AC'

J2	00	01	11	10
00		1		
01	1	1	X	X
11		1		
10		1	X	X

J2= C'D + A'B

K2	00	01	11	10
00	X	X	1	
01	X	X		1
11	X	X		
10	X	X		1

K2 = A'B'D + A'BD' + AB'D'

Y el diagrama logico:

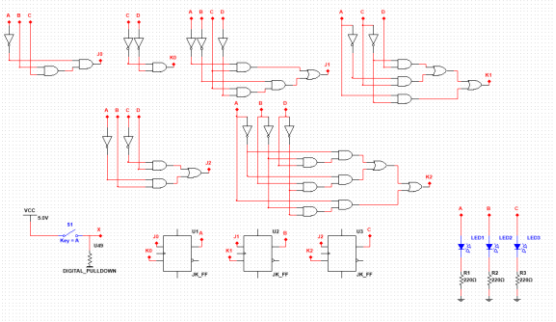


Diagrama Con Flip-flop tipo Jk

A continuacion, realizaremos el tipo SR, partiendo de su tabla de excitación:

S	R	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	?

(a) RS

Tabla de excitación del FF RS

FF tipo SR					
S2	R2	S1	R1	S0	R0
0	X	0	X	0	X
0	X	0	X	1	0
0	X	1	0	X	0
0	X	1	0	0	1
0	X	X	0	1	0
0	X	0	1	1	0
1	0	0	1	0	1
1	0	0	1	X	0
0	1	0	X	0	X
X	0	0	X	1	0
X	0	1	0	0	1
X	0	0	X	X	0
0	1	1	0	0	X
X	0	0	1	1	0

Sa	00	01	11	10
00				
01			1	1
11		X		
10		X	X	X

Ra	00	01	11	10
00	X	X	X	X
01	X	X		
11	1			
10	1			

Sb	00	01	11	10
00			1	1
01	X			
11	1			
10				1

Rb	00	01	11	10
00	X	X		
01		1	1	1
11		1		
10	X	X	X	

Sc	00	01	11	10
00		1		X
01	1	1	X	
11		1		
10		1	X	

Rc	00	01	11	10
00	X		1	
01				1
11	X			
10	X			1

Y el diagrama logico:

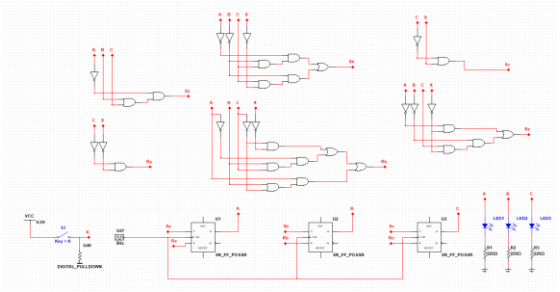


Diagrama con Flip-flop SR

Para finalizar, realizamos el tipo T, o toggle, para ello, partiremos de la tabla de excitación:

T	$Q(t+1)$
0	Q(t)
1	Q'(t)

(d) T

Tabla de excitación FF T

FF tipo T		
T2	T1	T0
0	0	0
0	0	1
0	1	0
0	1	1
0	0	0
0	1	0
1	1	1
1	1	0
1	0	0
0	0	1
0	1	1
0	0	0
1	1	0
0	1	1

T0	00	01	11	10
00				
01			1	1
11	1			
10	1			

T0 = AC'D' + A'BC

T1	00	01	11	10
00			1	1
01		1	1	1
11	1	1		
10				1

T1 = A'C + B'CD' + BC'D + ABC'

T2	00	01	11	10
00		1	1	
01				1
11		1		
10		1		1

T2 = A'B'D + AC'D + A'BCD' + AB'CD'

Y el diagrama logico es el siguiente:

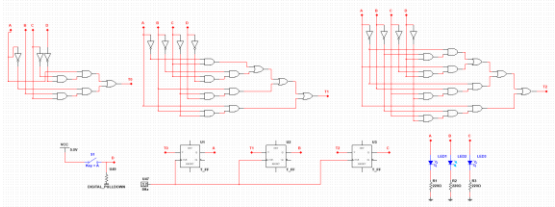


Diagrama con Flip-flop tipo T

V. **Conclusión**

Durante esta práctica se realizó el diseño, la simulación y la implementación física de un circuito secuencial utilizando diferentes tipos de flip-flops: D, JK, SR y T. El proceso comenzó con la elaboración de las tablas de estado y de transición, luego se llevó a cabo la simplificación de las funciones lógicas mediante mapas de Karnaugh y posteriormente se crearon los diagramas en Multisim para simular el comportamiento del circuito. Finalmente, se desarrolló el código en Verilog, incluyendo módulos personalizados y la configuración del reloj interno de la FPGA, con el propósito de comprobar el correcto funcionamiento de cada flip-flop durante los cambios de estado.